

SET1 A1. Анализ алгоритма SELECTION SORT

Инвариант P1: индекс минимального элемента в $A[i \dots j-1] = \text{minId}$

База: $\text{minId} = i, j = i+1$. Получается, что подмассив $A[i \dots j-1] = A[i]$ содержит один элемент $A[i]$, который и является минимумом в подмассиве из одного элемента

ПИ: предположим, что после $j = k$ итерации P1 выполняется

Шаг ($k \rightarrow k+1$): на шаге $j = k+1$ сравним $A[k]$ с $A[\text{minId}]$: если $A[k] < A[\text{minId}]$, то $\text{minId} = k$. Теперь minId указывает на минимум в $A[i \dots k]$, что соответствует $A[i \dots j-1]$ для $j = k+1$

Вывод: по принципу мат индукции P1 выполняется

Инвариант P2: подмассив $A[0 \dots i-1]$ содержит i наименьших элементов исходного массива, расположенных в отсортированном порядке

База: $i = 0$, подмассив $A[0 \dots i-1] = A[0 \dots -1]$ пуст, т.е. содержит 0 наименьших элементов и считается отсортированным

ПИ: предположим, что после $i = k$ итераций P2 выполняется

Шаг ($k \rightarrow k+1$): по доказанному из P1 находим минимум в $A[k \dots n-1]$, функция `swar` помещает этот минимум на позицию k . Тогда элемент $A[k] \geq$ всех из $A[0 \dots k-1]$ ($A[0 \dots k-1] \subseteq A[k-1 \dots n-1]$) и $\leq$ всех в $A[k+1 \dots n-1]$ (он минимум в $A[k \dots n-1]$). Получается, что $A[0 \dots k]$ - отсортирован и содержит $k+1$ наименьших элементов

Вывод: по принципу мат индукции P2 выполняется

P1:

```
void selectionSort(std::vector<int> &A) {
    size_t n = A.size();
    for (size_t i = 0; i < n; ++i) {
        // INIT:
        // minId указывает на первый элемент неотсортированной части
        int minId = i;

        // j = i+1, minId = i, подмассив A[i..j-1] = A[i..i] содержит один элемент A[i]
        // Единственный элемент подмассива является его минимальным элементом
        // Инвариант выполняется на этапе INIT
        for (size_t j = i + 1; j < n; ++j) {

            // MNT:
            // В начале каждой итерации j minId указывает на минимум в A[i..j-1]
            // Выполняется сравнение A[j] с текущим минимумом
            if (A[j] < A[minId]) {
                minId = j; // Обновляется minId, который указывает на минимум в A[i..j]
            }

            // После обновления или без него minId указывает на минимум в A[i..j]
            // Если A[j] < A[minId], то minId = j
            // Если A[j] >= A[minId], то minId остался прежним
            // Инвариант выполняется после каждой итерации
        }

        // TRM:
        // Условие выхода: j = n
    }
}
```

```

    // На последней итерации j = n-1, minId указывал на минимум в A[i..n-1]
    // Потом j = n, цикл завершается
    // minId содержит индекс минимального элемента в A[i..n-1]
    // Инвариант выполняется на этапе TRM
    std::swap(A[minId], A[i]);
}
}

P2:
void selectionSort(std::vector<int> &A) {
    size_t n = A.size();
    // INIT:
    // i = 0, подмассив A[0..i-1] = A[0..-1] является пустым
    // Пустой массив по определению: содержит 0 наименьших элементов и считается
отсортированным
    // Инвариант выполняется на этапе INIT

    for (size_t i = 0; i < n; ++i) {
        // MNT:
        // Подмассив A[0..i-1] содержит i наименьших элементов исходного массива и
отсортирован
        // Инвариант выполняется в начале итерации (по индукции)

        int minId = i;
        // Внутренний цикл находит минимальный элемент в неотсортированной части
A[i..n-1]
        for (size_t j = i + 1; j < n; ++j) {
            if (A[j] < A[minId]) {
                minId = j;
            }
        }

        // По инварианту P1 minId содержит индекс минимального элемента в
подмассиве A[i..n-1]
        // A[minId] является (i+1)-м наименьшим элементом во всём массиве, т.к.:
        // 1. По предположению индукции: A[0..i-1] содержит i наименьших элементов
        // 2. A[minId] - минимальный элемент в оставшейся части A[i..n-1]
        // Следовательно, A[minId] меньше или равен всем элементам в A[i..n-1]
        // и больше или равен всем элементам в A[0..i-1] (по построению)
        // Таким образом, A[minId] является (i+1)-м наименьшим элементом массива

        std::swap(A[minId], A[i]);

        // После swap:
        // A[i] содержит (i+1)-й наименьший элемент массива
        // Все элементы A[0..i-1] остаются на своих местах и отсортированы
        // A[i] >= всех элементов в A[0..i-1] и <= всех элементов в A[i+1..n-1]
        // A[0..i] содержит i+1 наименьших элементов и отсортирован
        // Инвариант сохраняется для следующей итерации
    }

    // TRM:

```

```
// Условие выхода -  $i = n$   
// Подмассив  $A[0..i-1] = A[0..n-1]$  - весь массив  
// Он содержит  $n$  наименьших элементов и отсортирован в порядке возрастания  
// Инвариант выполняется на этапе TRM  
}
```